Research Topics in Blockchain
&
Blockchain and Its Application

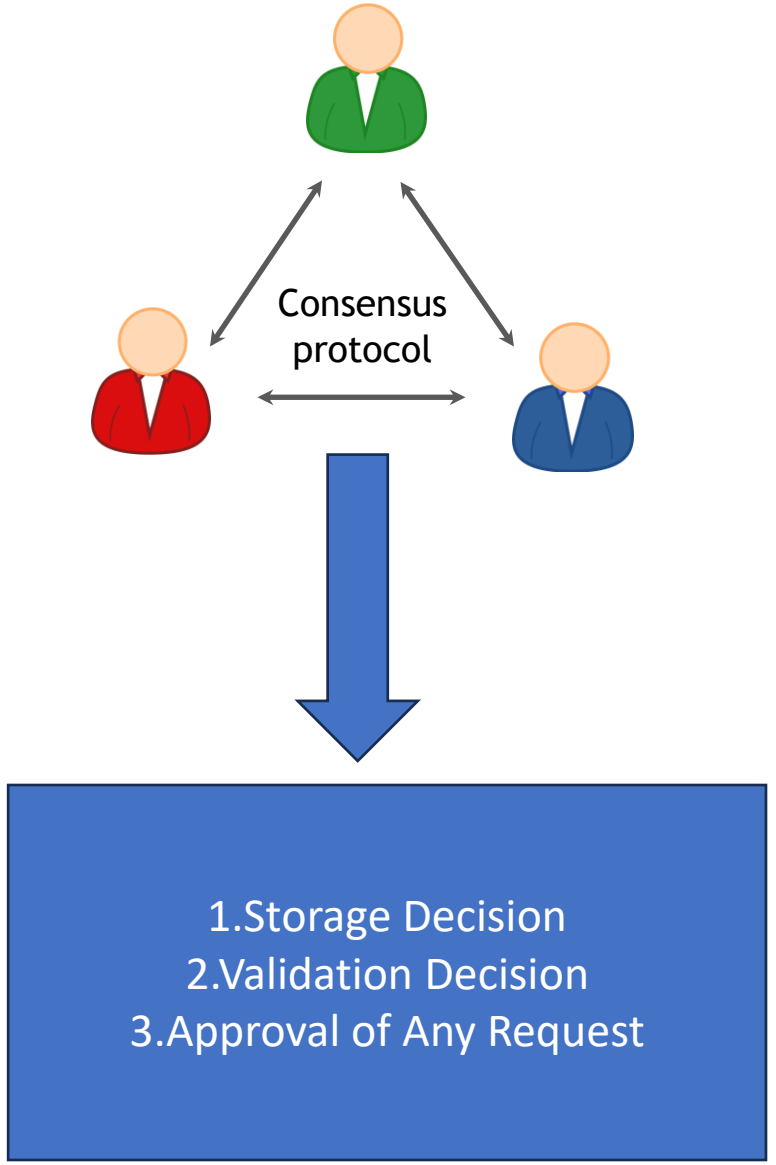


Lecture # 9:
Decentralization and Consensus Mechanism

- ✓ **Decentralization**
- ✓ **Distributed Decentralization**
- **Lab 2 Demonstration**

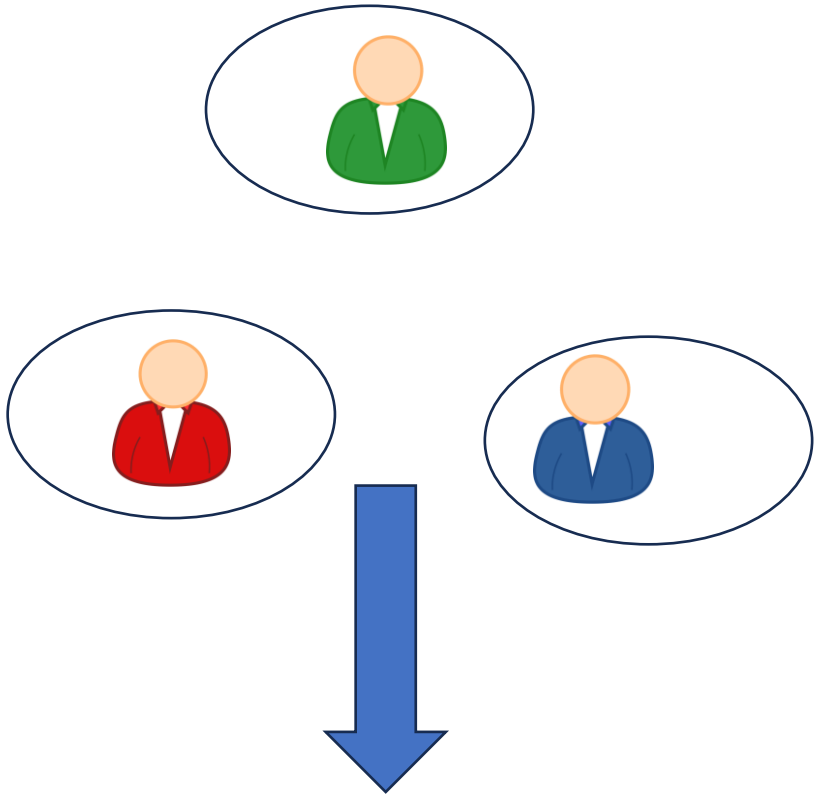
Distributed Decentralization

- ✓ Involves multiple independent nodes with no central authority
- ✓ Governed by consensus mechanisms. This structure emphasizes both distribution and decentralization, enhancing security and redundancy through distributed data storage.
- ✓ Examples include blockchain networks like Bitcoin and Ethereum, as well as decentralized file systems like IPFS and BitTorrent.

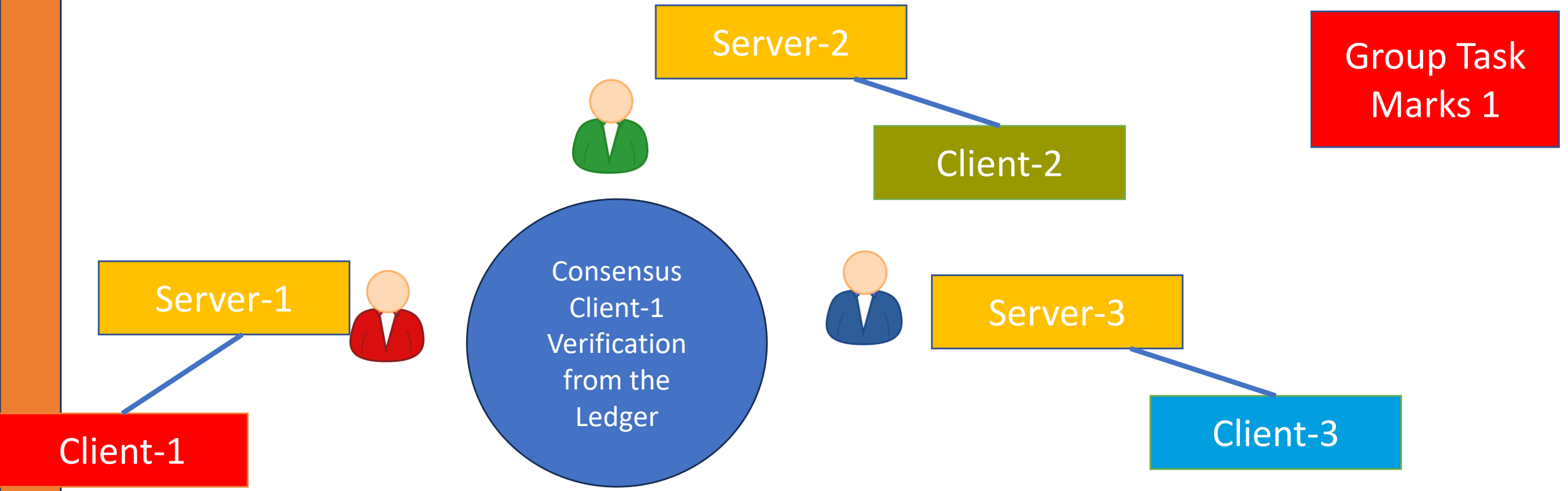


Decentralization

- ✓ Refers to the spreading out of control across multiple entities, though it may still retain some central oversight.
- ✓ It's less about independent nodes and more about shifting authority from a central body to a distributed set of entities.
- ✓ For instance, in multi-cloud infrastructure or federated governance models, data and control are distributed, but coordination might still be centralized in some instances.



Implementation Distributed Decentralized Network For Consensus Mechanism For Recording Hello Message in Ledger



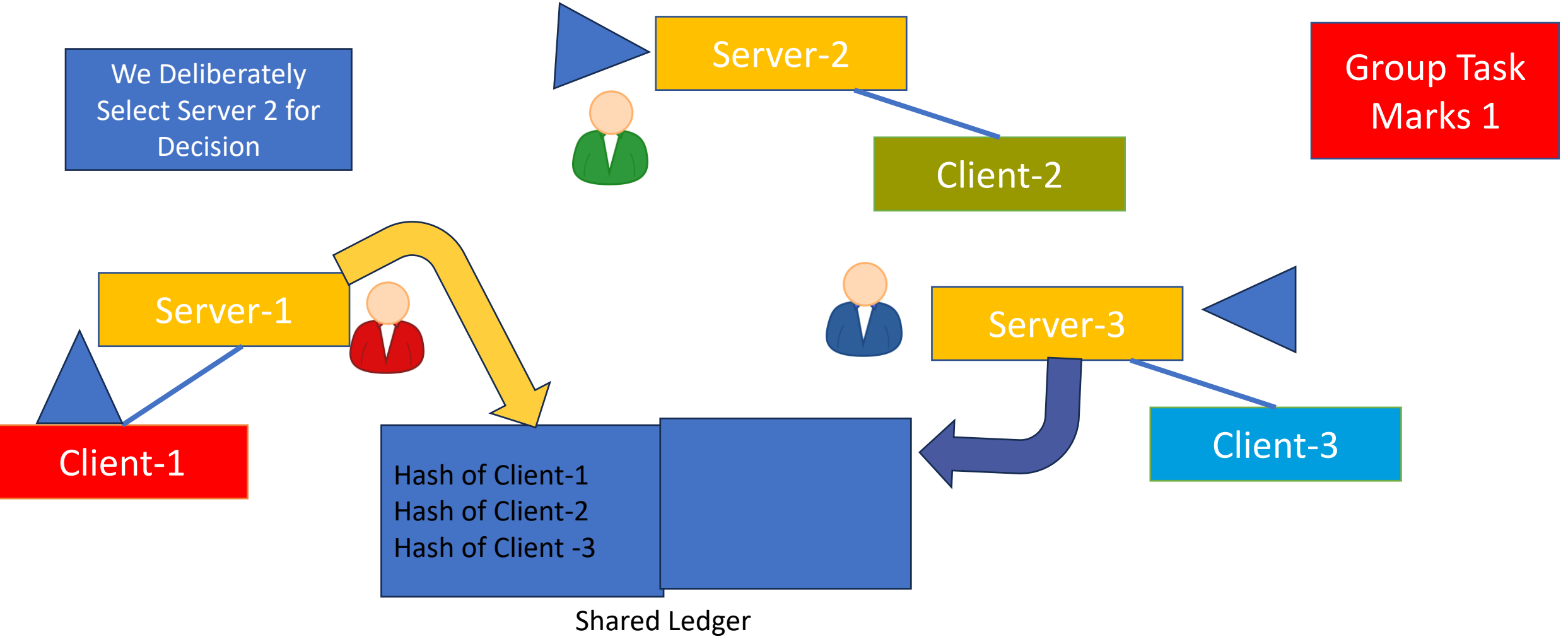
Implementation Distributed Decentralized Network For Consensus Mechanism For Recording Hello Message in Ledger

How can we implement a **distributed decentralized network** for a **consensus mechanism solution**, where three servers, each having their own clients, share a single **JSON-based ledger** containing client identity hashes for consensus?

Specifically, one of the servers is deliberately selected to store transactions and validate client hash requests by verifying them from the ledger.

For example, suppose **Client 1 from Server 1** sends a "Hello" message. How should the consensus mechanism handle the verification and validation of this transaction across the network?
What are the best practices and potential challenges in designing such a system to ensure security, integrity, and fault tolerance?

Implementation Distributed Decentralized Network For Consensus Mechanism Solution



Research Topics in Blockchain
&
Blockchain and Its Application



Lecture # 10:
Decentralization and Consensus Mechanism

- ✓ Decentralization
- ✓ Distributed Decentralization
- ✓ Consensus Mechanism
- ✓ Consensus without identity
- ✓ Consensus Algorithm
- ✓ Implicit Consensus algorithm

Decentralization in Bitcoin

Peer-to-peer network:

For Digital Transaction

Minner Nodes:

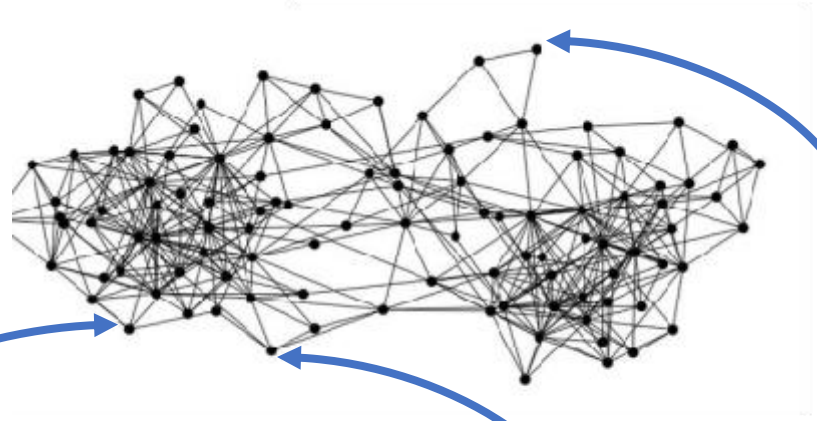
a system that allows **anyone to participate**, but over time, **power naturally accumulates** in the hands of a few entities, which is often perceived as a **problem** or **undesirable outcome**.



Perform the role of responsible entities to forward the transaction and validation of security attributes

✓ What is Distributed Decentralization

Use-Case of Transaction A->B



Verify the Transanction

Verify the Transanction

Verify the Transanction

Decision will be taken on the Basis of there verification (Discuss the Strategies)

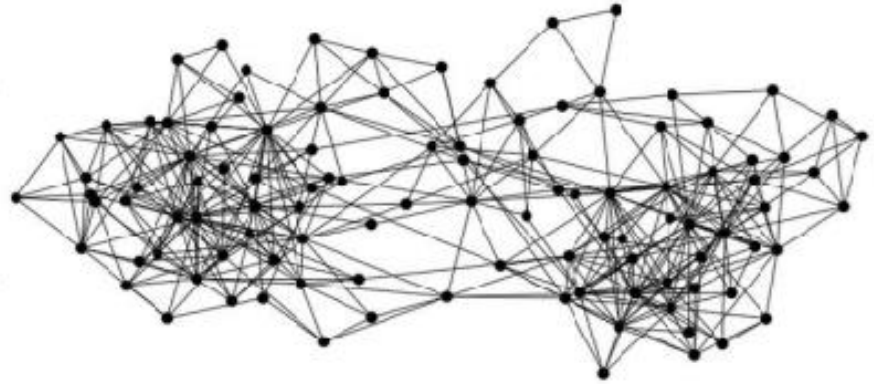
Distributed consensus protocol. There are *n* nodes that each have an input value. Some of these nodes are faulty or malicious. A distributed consensus protocol has the following two properties:

- It must terminate with all honest nodes in agreement on the value
- The value must have been generated by an honest node



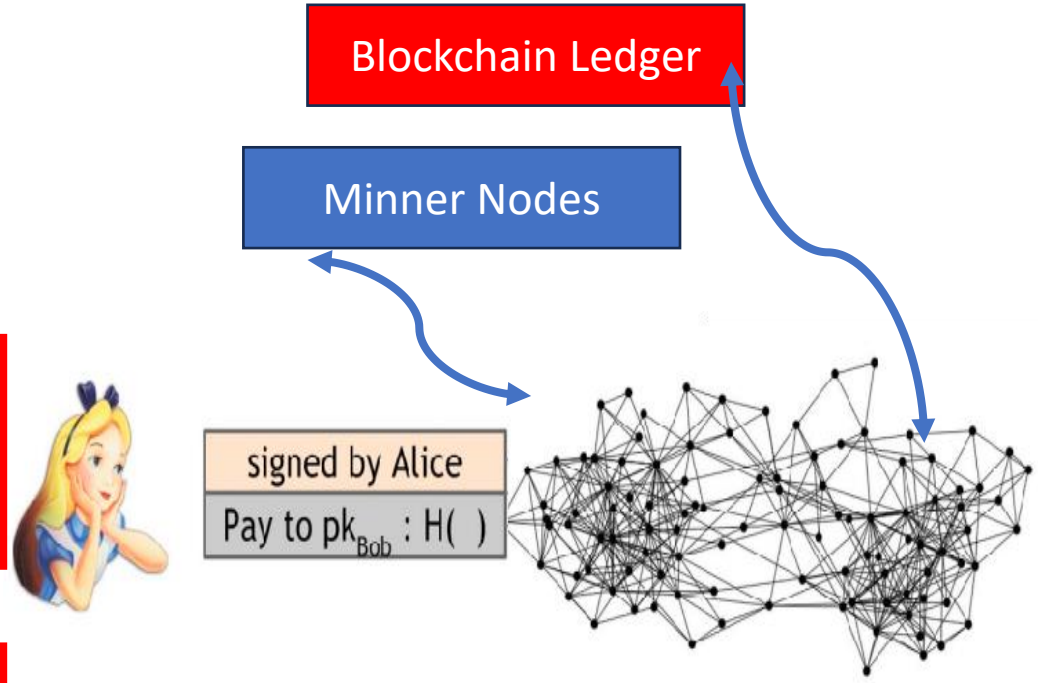
signed by Alice

Pay to pk_{Bob} : H( )



Alice broadcasts the transaction to the entire Bitcoin peer-to-peer network.

- Alice broadcasts the transaction to all the peer-to-peer nodes,
- Bob is running one of the nodes in the peer-to-peer network to broadcast the transaction
- Will Alice be the legitimate user of the network
- Will this transaction is valid or not

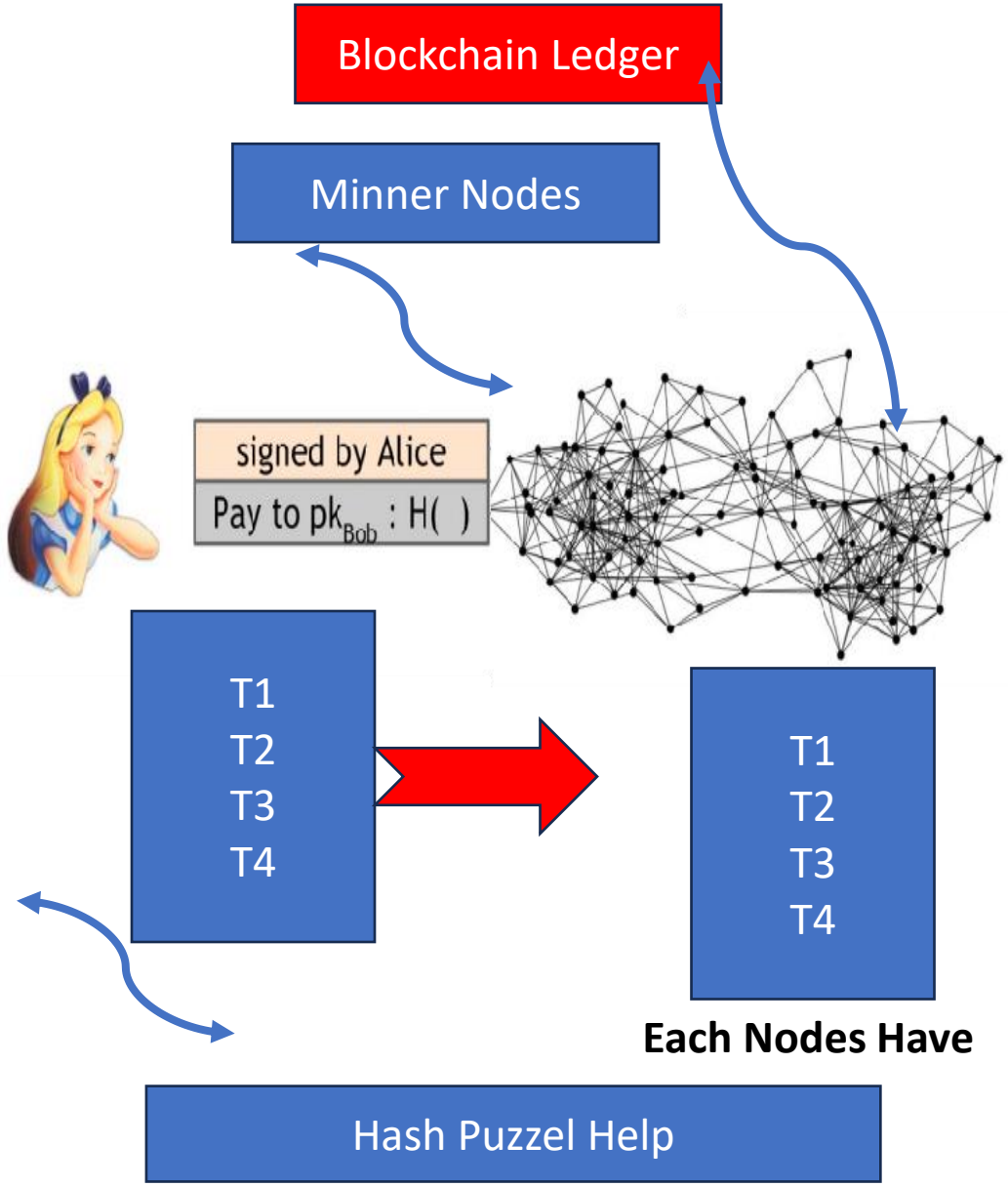


Minner Nodes Store in Blockchain, Alice Signature

What other things is consider in bitcoin network ?

Alice broadcasts the transaction to the entire Bitcoin peer-to-peer network.

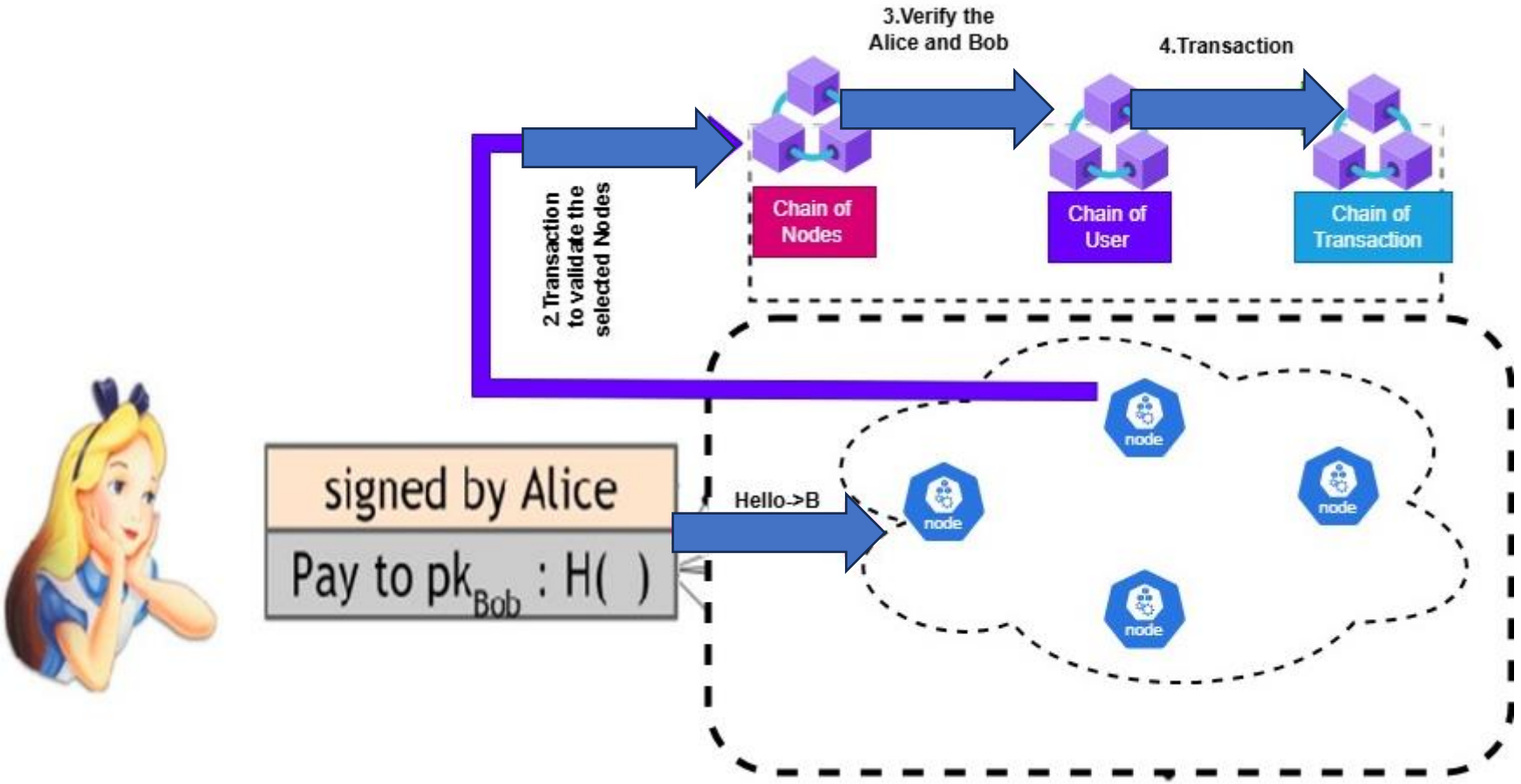
- Each Node in the network has their own queue mechanism
- Pool of outstanding transactions that it has hea about but have not yet been included on the block chain.
- For consensus distributed decentralized entitie already selected
- Avoid race conditions
- Nodes have a common mechanism to put the transaction to the network and inform all the nodes in the network



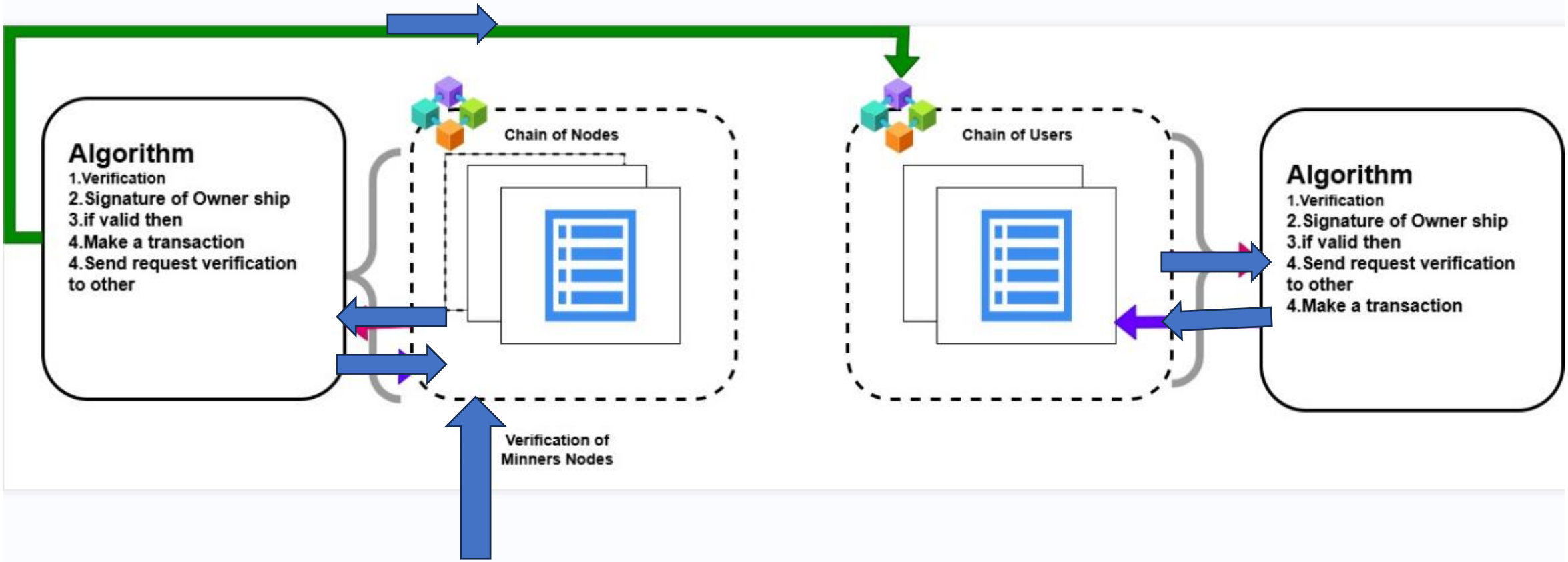
Consensus algorithm (simplified)

1. New transactions are broadcast to all nodes
2. Each node collects new transactions into a block
3. In each round a random node gets to broadcast its block
4. Other nodes accept the block only if all transactions in it are valid (unspent, valid signatures)
5. Nodes express their acceptance of the block by including its hash in the next block they create

Alice transactions in Bitcoin Network



WPS Photos Shahbaz New -Page-74.jpg 83.63 KB , 1397 × 416 px , 1/17
Upgrade Now
AI Tools Edit Convert Print



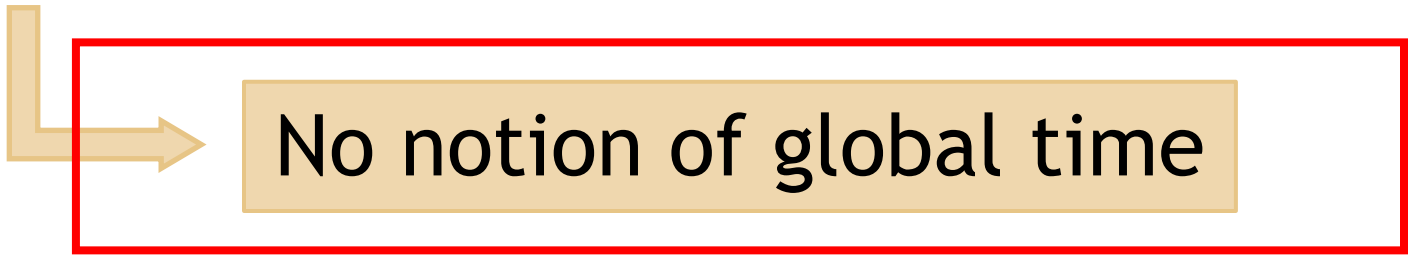
Search
10:17 AM
2/9/2025

Why consensus is hard

Nodes may crash
Nodes may be malicious

Network is imperfect

- Not all pairs of nodes connected
- Faults in network
- Latency



No notion of global time

No Global Time

- ✓ In a distributed system, nodes (computers) are located in different places and communicate over a network.
- ✓ Because network latency (delay in data transmission) is **high and unpredictable**, different nodes may **receive messages at different times**.
- ✓ This means there is **no single, universal clock** that all nodes can rely on to determine the exact order of events.

No Agreement on Event Ordering

- ✓ Since nodes receive messages at different times, they **cannot rely on timestamps** to agree on the order in which events (like transactions or messages) occurred.
- ✓ For example, if Node A sees that Message 1 arrived first and then Message 2, but Node B sees the opposite, they will **disagree on the event order**.

Why consensus is hard

Why consensus is hard

Impossibility Results: Several results in the literature prove that achieving consensus in certain situations is **impossible**. These results arise due to constraints that are inherent to distributed systems:

Byzantine Generals Problem: This is a classic problem that illustrates the challenge of achieving consensus in the presence of faulty (or "traitorous") nodes. The scenario describes an army of generals who need to agree on a battle plan, but some generals may intentionally try to deceive the others. The impossibility result here states that if up to one-third of the generals are traitors, it is impossible for the loyal generals to reach a unified decision.

Fischer-Lynch-Paterson Impossibility: This result is more subtle and concerns the situation where even a single faulty process can prevent consensus. Under certain conditions (such as deterministic behavior by nodes), the authors showed that consensus is **impossible** if there is even one faulty process in the system.

```python
import hashlib
import time

def proof_of_work(data, difficulty=2):
    """
    A simple Proof of Work function that finds a nonce
    such that the hash has 'difficulty' leading zero pairs.

    :param data: The transaction data or block data to be hashed
    :param difficulty: Number of leading zero pairs (e.g., difficulty=2 means "0000" a
    :return: (nonce, hash_value, time_taken)
    """
    nonce = 0
    prefix = '0' * (difficulty * 2)# Two zeros per difficulty level
    print(prefix)
    start_time = time.time()

    while True:
        text = f"{data}{nonce}".encode()
        hash_value = hashlib.sha256(text).hexdigest()
        print(hash_value)

        if hash_value.startswith(prefix):
            end_time = time.time()
            return nonce, hash_value, round(end_time - start_time, 4)  # Return the nonce, hash, and

        nonce += 1  # Increment nonce and retry
# Example Usage
data = "Shahbaz"
difficulty = 2  # Require three pairs of leading zeros (i.e., "000000")
nonce, final_hash, time_taken = proof_of_work(data, difficulty)

print(f"✅ Proof of Work Completed!")
print(f"Nonce Found: {nonce}")
print(f"Hash: {final_hash}")
print(f"Time Taken: {time_taken} seconds")
```

1.Set the Nounce =0
The with Prefix =set the Difficulty of finding Zero Pairs
Here in the code it is set as 0000

Now When the Data Shahbaz is enter

In this example Shahbaz String is not throw next time inorder to maintain the Concept of randomness

Bitcoin consensus: theory & practice

Bitcoin consensus works better in practice than in theory

Theory is still catching up

BUT theory is important, can help predict unforeseen attacks

Some things Bitcoin does differently

Introduces incentives

- Possible only because it's a currency!

Embraces randomness

- Does away with the notion of a specific end-point
- Consensus happens over long time scales — about 1 hour

Bitcoin and Randomness: Bitcoin's consensus algorithm, known as **Proof of Work (PoW)**, heavily relies on randomness, particularly in the mining process. Miners must solve complex cryptographic puzzles, which essentially involves a random trial-and-error process. This randomness is key to the security and decentralization of the Bitcoin network. Each miner has an equal chance of solving the puzzle, and the winner gets to propose the next block.

No Fixed Starting or Ending Point: Unlike traditional consensus protocols, Bitcoin doesn't have a well-defined "start" and "end" point for achieving consensus. Instead, **consensus happens gradually** over time. Practically, it may take about an hour for a transaction to be included in the blockchain with a high degree of certainty, but there's always a degree of uncertainty during this process.

Increasing Probability of Consensus Over Time: Initially, there's a chance that a transaction or block may not be accepted by the majority of nodes in the network. However, as more blocks are added to the chain after a particular block, the likelihood that the block will become part of the **final consensus** increases. Over time, the probability that your view of the block will match the **eventual consensus view** increases exponentially. Conversely, the likelihood of divergent views (i.e., forks) decreases as more blocks are added.

Consensus without identity: the block chain

Without Identity → Unlike traditional systems where users must be identified (e.g., banks, government databases), blockchain allows **anonymous or pseudonymous participation**.

Bitcoin prevents Sybil attacks without relying on a central authority

No Central Authority → Unlike traditional systems, Bitcoin has no central body to assign or verify node identities.
Sybil Attack Risk → Attackers could create multiple fake nodes to manipulate consensus.

1. **Pseudonymity by Design** → Bitcoin ensures that participants don't need to reveal real-world identities.
2. **Security Challenges Without Identities** → Traditional consensus could rely on known identities, but Bitcoin can't.
3. **Random Selection Approach** → Bitcoin compensates for the lack of identities by using a **lottery-like system** (Proof of Work) to randomly select a node for mining.
4. **Preventing Sybil Influence** → Even if an attacker creates many fake nodes, they **cannot increase their control** over the network beyond their actual computational power.

Main Takeaway:
Bitcoin replaces **identity-based trust** with **computational proof (PoW)** to ensure fairness and prevent Sybil attacks. 🚀

Mathematical Representation of a Sybil Attack

Let:

- N = Total number of nodes in the network
- M = Honest nodes
- S = Sybil nodes (fake identities created by an attacker)
- f = Fraction of the network controlled by the attacker

We define:

$$f = \frac{S}{M + S}$$

If f approaches 1, the attacker gains full control over the network.

In a **voting-based consensus mechanism**, if the attacker has more than **50% of the network**, they can **override honest decisions.** This occurs when:

S>M

which means the attacker has a majority.

Bitcoin achieves consensus without explicit voting or agreement mechanisms through **"implicit consensus."**

1. **Random Node Selection** → Each round (block creation), a node is randomly chosen to propose the next block.
2. **No Traditional Consensus Algorithm** → Unlike traditional systems, there is **no voting** or explicit agreement process for selecting blocks.
3. **Unilateral Block Proposal** → The selected node proposes a block, whether honest or malicious.
4. **Implicit Validation by Network** → Other nodes **decide** whether to accept or reject the block **by building on it** (acceptance) or **ignoring it** (rejection).
5. **Technical Mechanism** → Since each block **hashes the previous block**, nodes signal which chain they support simply by extending it.

Research Topics in Blockchain
&
Blockchain and Its Application



Lecture # 11:
Why this Consensus mechanism Works

✓ Why Consensus Algorithm works

Use case Stealing Bitcoin
Denial of Service Attack
Double Spend Attack

✓Incentive and Proof of work

✓Putting it all to geather

Usecase-1

Bitcoin’s consensus algorithm relies on cryptographic security and decentralized validation to prevent malicious activities. If an adversary, Alice, tries to manipulate the system, she faces strict cryptographic barriers.

1.Signature Verification Ensures Ownership

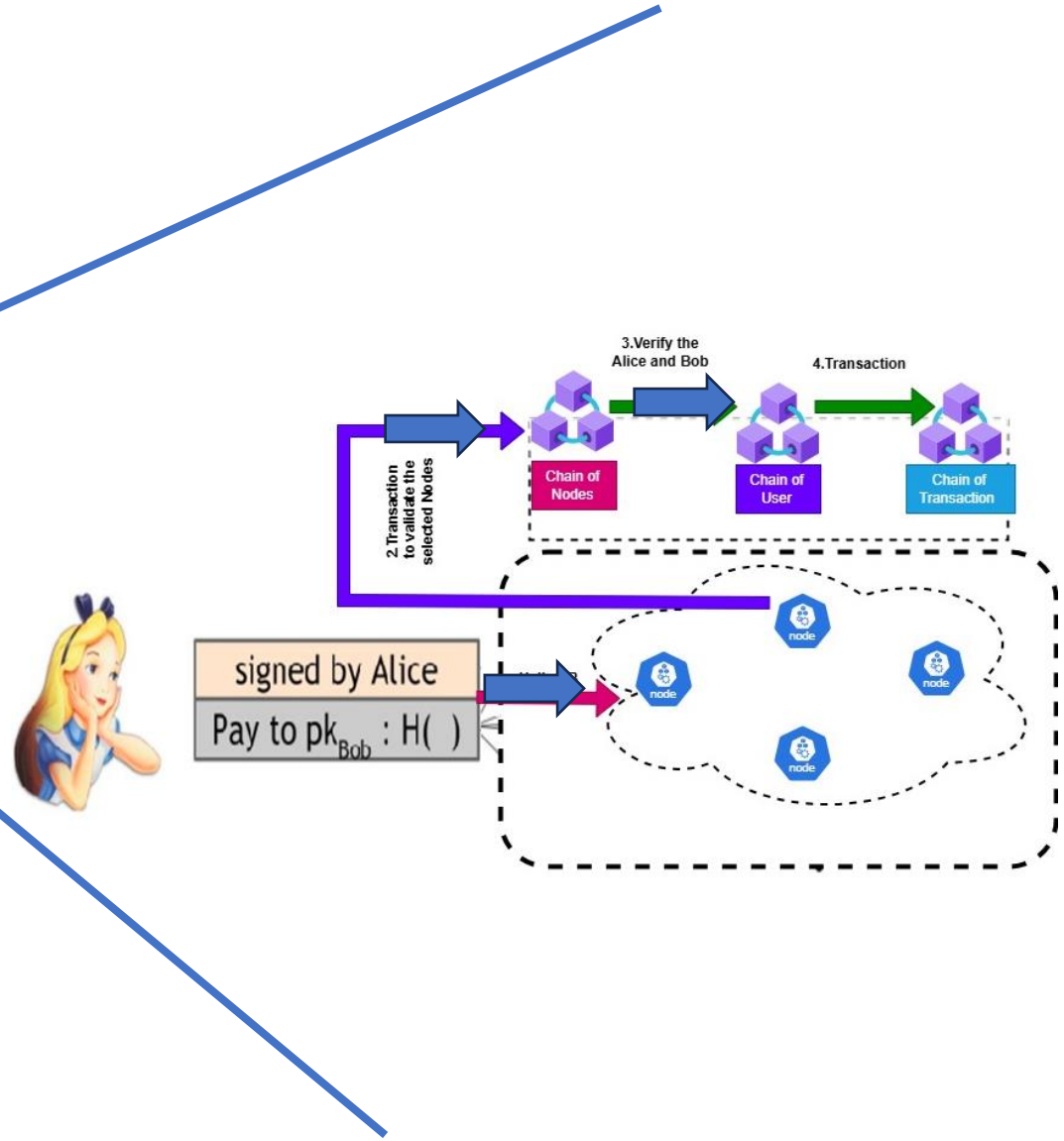
1. Every Bitcoin transaction requires a **valid digital signature** from the rightful owner.
2. Even if Alice is chosen to propose the next block, she cannot forge another user’s signature.

2.Decentralized Validation Blocks Fraud

1. All transactions must be verified by the network before inclusion in the blockchain.
2. Even if Alice attempts to include a fraudulent transaction, honest nodes will reject it.

3.Immutable Ledger Prevents Unauthorized Changes

1. Once a transaction is confirmed and added to the blockchain, it **cannot be altered or reversed** without controlling the majority of computational power (51% attack).
2. This ensures that Alice cannot retroactively change ownership records.



Mitigates Denial-of-Service Attacks

Bitcoin's consensus mechanism ensures that no single entity can permanently block transactions, even if a malicious actor like Alice attempts a **Denial-of-Service (DoS) attack** against another user, Bob.

1.Block Proposers Rotate Randomly

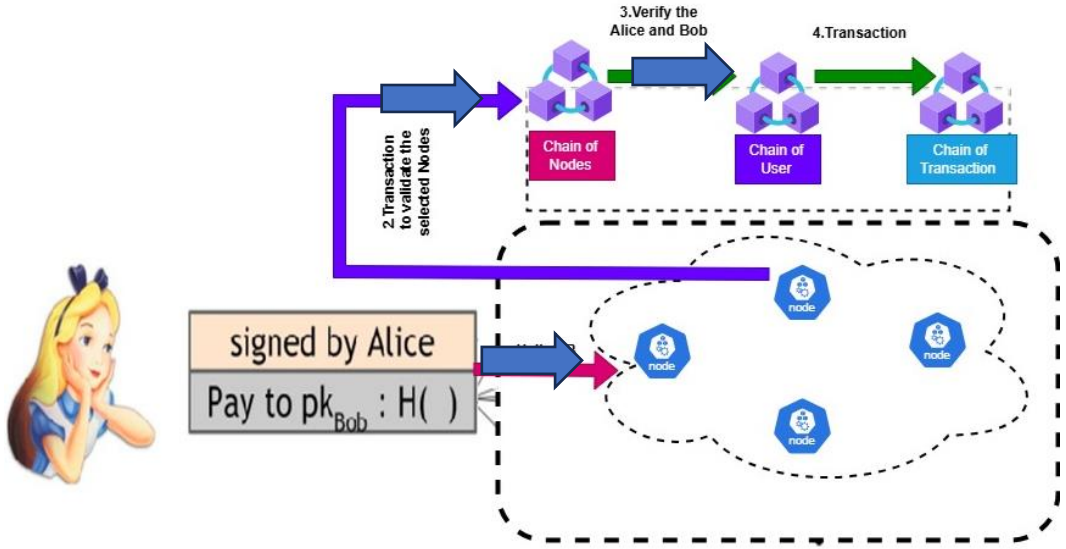
1. Alice might refuse to include Bob's transactions in her proposed block.
2. However, the **next block proposer is randomly selected**, meaning Bob's transaction will be included by another honest node.

2.Network Redundancy Prevents Censorship

1. Bitcoin's decentralized nature ensures that multiple nodes validate transactions.
2. Even if Alice ignores Bob's transaction, it will remain in the **mempool** until a fair miner includes it.

3.Transaction Persistence Ensures Inclusion

1. Transactions **do not expire** and remain valid until confirmed.
2. Eventually, Bob's transaction will be processed, making Alice's attack ineffective.



Double-Spend Attack

- **Alice's Initial Transaction:**
  - Alice buys software from Bob and sends a Bitcoin transaction to his address.
  - An honest node includes this transaction in the blockchain.
- **Bob Assumes Payment is Confirmed:**
  - Bob sees the transaction in the blockchain and allows Alice to download the software.
- **Alice's Attack (Double Spend Attempt):**
  - If Alice is selected to propose the next block, she creates a new block **ignoring** the one with her payment to Bob.
  - Instead, she includes a transaction sending the same bitcoins to another address she controls.

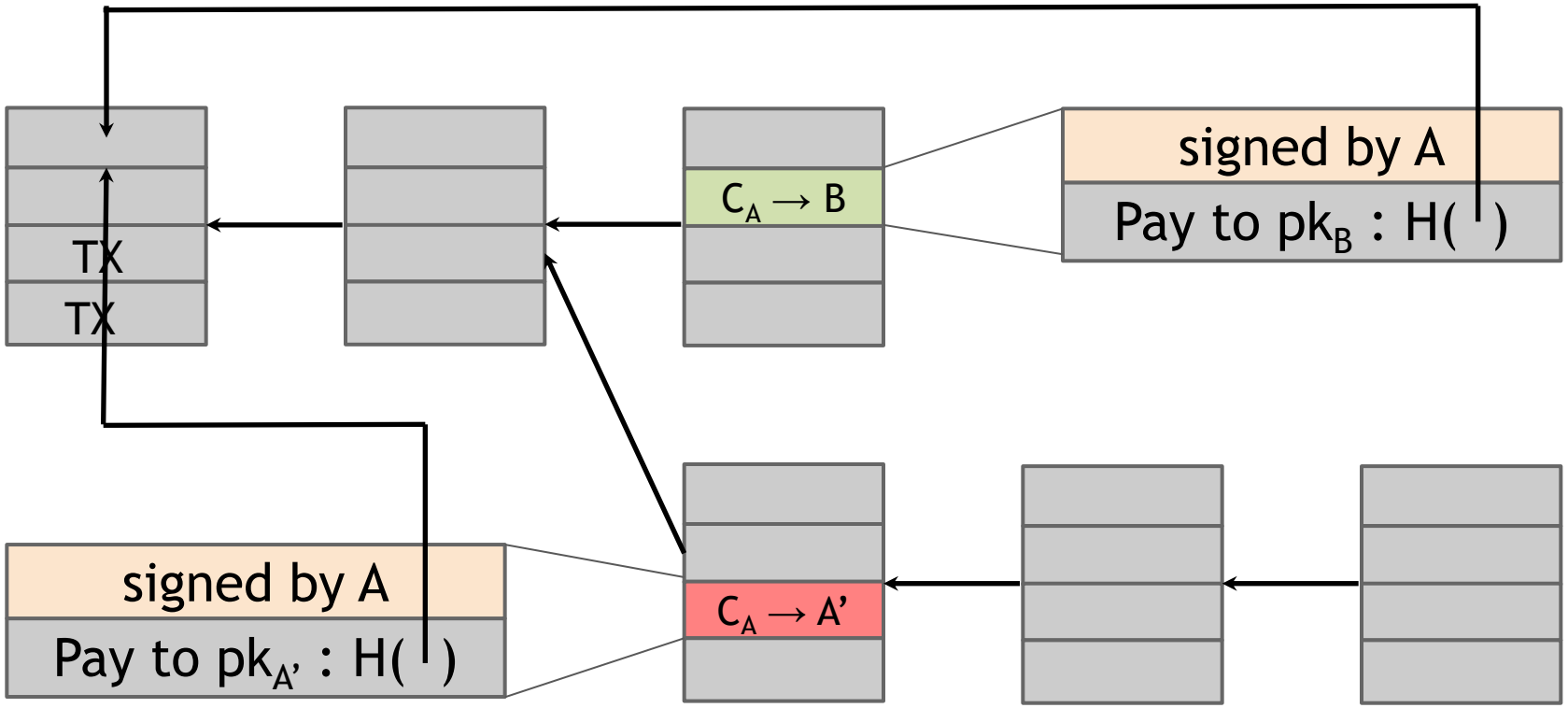
Honest Chain (longest valid chain) Block 50 → Block 51 (Txn: Alice → Bob) → Block 52

Malicious Chain (Alice's forked chain) Block 50 → Block 51' (Txn: Alice → Alice) → Block 52'

- **Conflict in the Blockchain:**
  - Two competing chains exist:
    - One where Alice pays Bob.
    - One where Alice pays herself.
  - Since both chains are the same length, the network can extend either one.
- **Outcome Depends on Network Consensus:**
  - The longest valid chain is accepted by honest nodes.
  - If honest nodes extend Bob's chain, Alice's attack fails.
  - If they extend Alice's chain, Bob never gets paid, and Alice successfully double-spends

- If the **honest chain grows longer**, the network **ignores Alice's forked chain** and follows the longest valid chain.
- If **Alice controls 51% of miners**, she can extend her fraudulent chain faster, making it the longest and causing Bob's transaction to disappear.

What can a malicious node do?



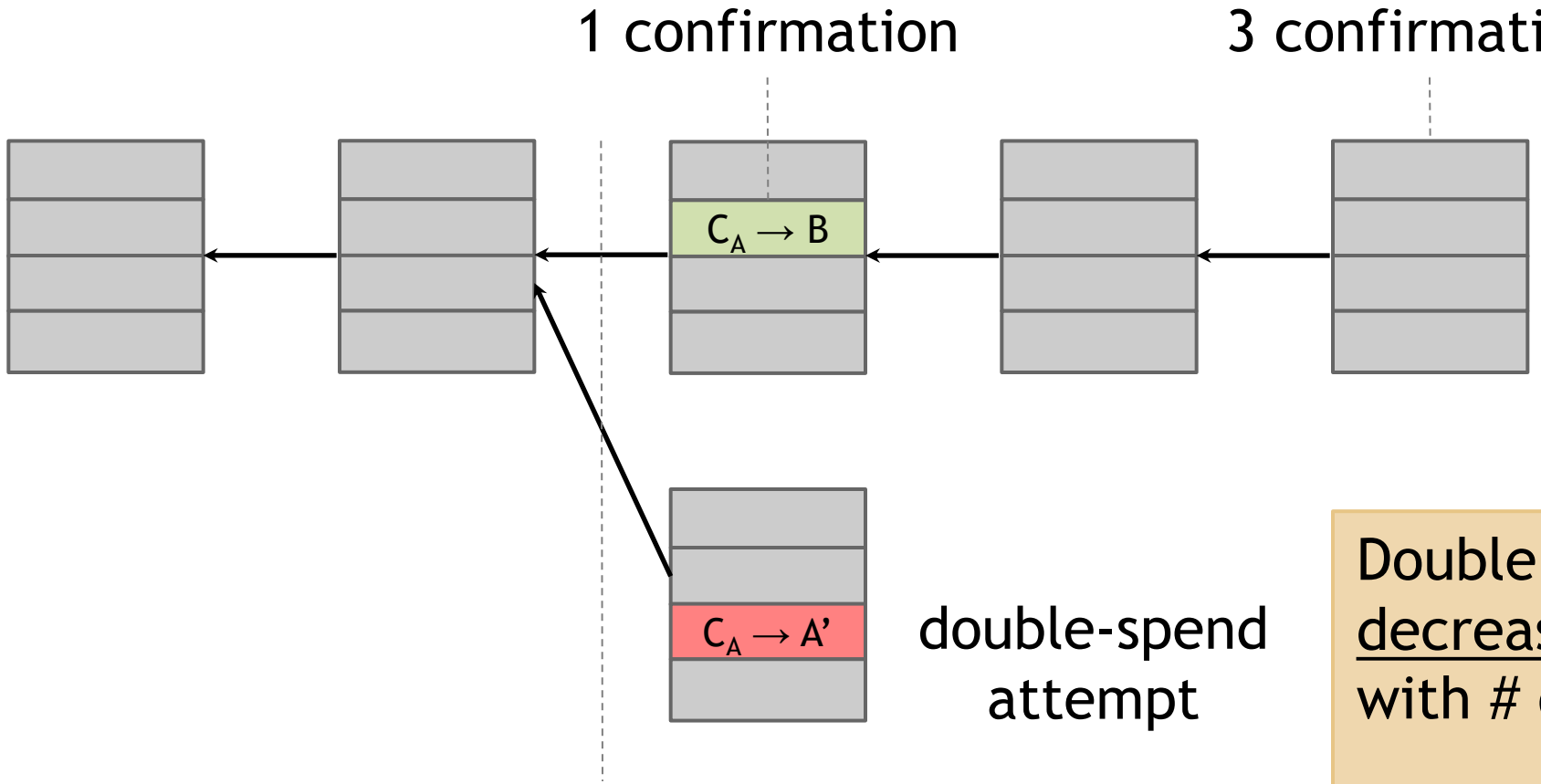
Double-spending attack

TX

Honest nodes will extend the longest valid branch

How this stop ??

From Bob the merchant's point of view



Double-spend probability <u>decreases exponentially</u> with # of confirmations

Most common heuristic:
6 confirmations

Alice Sends Bitcoin to Bob & Becomes the Miner

Step 1: Alice Creates a Transaction

- Alice wants to send 1 BTC to Bob.
- She signs the transaction with her private key and broadcasts it to the network.
- The transaction enters the mempool (waiting area for unconfirmed transactions).

Step 2: Proof-of-Work (PoW) Begins

- All miners in the network compete to solve a cryptographic puzzle.
- The miner who solves it first gets to add a new block to the blockchain.
- In this case, Alice wins the mining race and gets to propose the next block. attempt unless she controls over 51% of the mining power, which is highly unlikely.

Step 3: Alice Mines the Block

- Since Alice is the miner, she includes her own transaction (Alice → Bob, 1 BTC) in the new block.
- She solves the PoW puzzle, attaches the block to the longest valid chain, and broadcasts it.
- Other nodes in the network receive the block and verify:
  - Is the PoW valid? ✅ Are all transactions legitimate? ✅

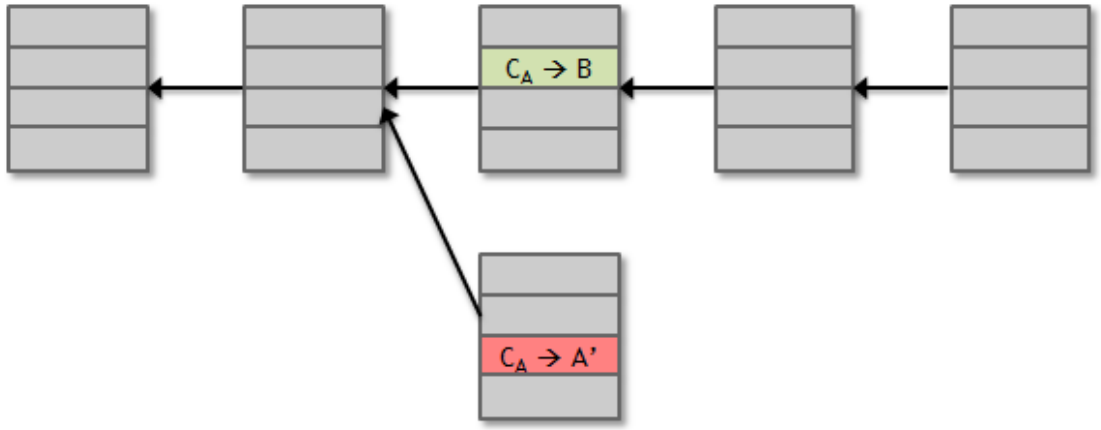
Step 4: Network Confirmation Process

- Other nodes start building on Alice's block, extending the blockchain.
- As new blocks are added, Bob waits for 6 confirmations before fully trusting the transaction.
- After 6 confirmations, Bob is sure that the transaction is final and irreversible.

Potential Attack: Alice Tries to Double Spend

- If Alice is dishonest, she could try to create another transaction sending the same 1 BTC to herself instead of Bob.
- However, for this attack to succeed, Alice must:
  - Mine a competing chain faster than the honest network.
  - Convince other miners to follow her fraudulent chain.
- If honest miners keep following the longest valid chain (which includes the transaction to Bob), Alice's double-spend attempt fails.

Recap



Protection against invalid transactions is cryptographic, but enforced by consensus

Protection against double-spending is purely by consensus

You're never 100% sure a transaction is in consensus branch. Guarantee is probabilistic